

Principles for Reliable, Auditable, and Cost-Aware LLM Pipelines

Table of contents

1	Request Batching	3
1.1	Description	3
1.2	Requirements	3
1.3	Example	4
2	Dry-Run Execution	4
2.1	Description	4
2.2	Requirements	4
2.3	Example	5
3	Preflight Validation and Cost Estimation	5
3.1	Description	5
3.2	Requirements	5
3.3	Example	6
4	Human-in-the-Loop Execution Gate	6
4.1	Description	6
4.2	Requirements	7
4.3	Example	7
5	Execution Provenance	7
5.1	Description	7
5.2	Requirements	7
5.3	Example	7
6	Programmatic Output Validation	7
6.1	Description	7
6.2	Requirements	8

6.3	Example	8
7	Deterministic–Stochastic Separation	8
7.1	Description	8
7.2	Requirements	8
7.3	Example	8
8	Stagewise Task Decomposition	8
8.1	Description	8
8.2	Requirements	9
8.3	Example	9
9	Checkpointing	9
9.1	Description	9
9.2	Requirements	9
9.3	Example	9
10	Resumable Execution	9
10.1	Description	9
10.2	Requirements	10
10.3	Example	10
11	Dependency-Aware Incremental Processing	10
11.1	Description	10
11.2	Requirements	10
11.3	Example	10
12	Atomic Persistence	10
12.1	Description	10
12.2	Requirements	11
12.3	Example	11
13	Artifact Immutability	11
13.1	Description	11
13.2	Requirements	11
13.3	Example	11
14	Failure Observability	11
14.1	Description	11
14.2	Requirements	12
14.3	Example	12
15	Completion Reporting	12
15.1	Description	12

15.2 Requirements	13
15.3 Example	13
16 Explicit Function Provenance	13
16.1 Description	13
16.2 Requirements	13
16.3 Example	14
17 Standardized Script Presentation	14
17.1 Description	14
17.2 Requirements	14
17.3 Example	15
18 Explicit Script I/O Contracts	15
18.1 Description	15
18.2 Requirements	15
18.3 Example	15

LLM pipelines become more reliable when model calls are incorporated into a larger software system. Here’s a summary of the principles I usually follow when creating a programming routine involving generative AI.

The following principles emphasize managing costs, maintaining evidence, validating results, recovering from errors, and separating the unpredictable nature of models from deterministic processes. These safeguards help ensure high-quality work assisted by LLMs.

1 Request Batching

1.1 Description

Request batching groups multiple processing units into bounded API requests. The goal is to reduce unnecessary API calls without creating requests that are too large, difficult to validate, or harmful to model performance.

1.2 Requirements

Batch as many compatible items as is practical while enforcing limits such as item count, serialized input size, context-window size, expected output size, and validation constraints.

1.3 Example

Instead of sending 20 paragraphs through 20 separate API calls, send the 20 paragraphs together when they fit safely within the request limits and can still be validated individually.

2 Dry-Run Execution

2.1 Description

A dry run exercises the pipeline without making consequential API calls. It lets the user inspect what *would* happen before spending money or creating external side effects.

2.2 Requirements

A dry run should resolve and display the planned model, processing scope, request count, important parameters, resource estimates, and output destination while preventing live API execution.

```
---

DRY RUN
No API requests will be sent.

Model
  Model:          gpt-5-nano
  Reasoning effort: medium
  Temperature:    unspecified

Scope
  Stage:          06B
  Input:          output/stage06a.json
  Processing units: 746
  Reusable results: 700
  New items:      46

Requests
  Planned requests: 3
  Batch limit:     20 items
  Largest batch:   20 items
```

```
Estimated resources
Input tokens:      ~72,000
Output tokens:     ~12,000
Total tokens:      ~84,000
Incremental cost:   ~$0.18

Output
Destination:       output/stage06b/
Existing artifacts: preserved

DRY RUN COMPLETE
No API requests were sent.

---
```

2.3 Example

Running `--dry-run` reports that 12 requests would be sent, which model would be used, and the estimated token consumption, but sends no requests.

3 Preflight Validation and Cost Estimation

3.1 Description

Preflight checks determine whether a planned run is valid and estimate its resource requirements before execution. In resumable pipelines, the estimate should focus on work that still requires new inference.

3.2 Requirements

Before execution, identify new requests, estimate token consumption and incremental monetary cost after accounting for reusable results, and stop the run if configured operational limits would be exceeded.

3.3 Example

A preflight report might say:

```
---  
  
PREFLIGHT CHECK  
  
Work  
  Processing units:      746  
  Reusable results:     700  
  New items:            46  
  Planned requests:      3  
  
Estimated resources  
  Input tokens:          ~72,000  
  Output tokens:         ~12,000  
  Total tokens:          ~84,000  
  Incremental cost:      ~$0.18  
  
Configured limits  
  Token ceiling:         100,000  
  Spending ceiling:      $1.00  
  
PREFLIGHT PASSED  
Execution is within configured limits.  
  
---
```

If the run exceeds a configured token or spending ceiling, execution is blocked.

4 Human-in-the-Loop Execution Gate

4.1 Description

An execution gate ensures that consequential API calls happen only after explicit human authorization. Safe behavior should be the default.

4.2 Requirements

Live execution must require an affirmative action, and the guard should be enforced at the consequential execution boundary where practical rather than relying only on a user-interface convention.

4.3 Example

A command requires `--execute` before making API calls. The API-calling function also refuses to run unless it receives an explicit execution authorization.

5 Execution Provenance

5.1 Description

Execution provenance preserves enough information to understand, audit, and reconstruct what happened during each LLM invocation. Raw model responses are primary evidence and should not disappear when parsing fails.

5.2 Requirements

Record model identity, relevant parameters, token usage, timestamps, prompt and configuration versions, hashes or fingerprints, and request/response artifacts. Preserve the authoritative raw model response before lossy parsing, normalization, or transformation.

5.3 Example

For each call, save the raw response first and then the parsed result, together with the model snapshot, prompt hash, timestamp, token usage, and request ID.

6 Programmatic Output Validation

6.1 Description

LLM output should not be trusted merely because it looks plausible or satisfies a generation-time schema. Independent code should verify the properties that downstream processing depends on.

6.2 Requirements

Use generation-time constraints when useful, but independently enforce structural schemas, allowed values, logical consistency, and source-grounded invariants before accepting an output.

6.3 Example

If the model returns a quotation as supporting evidence, code verifies that the quotation is an exact substring of the source paragraph. If it is not, the output is rejected or flagged.

7 Deterministic–Stochastic Separation

7.1 Description

LLMs are useful for tasks requiring semantic judgment, but many pipeline operations are better handled by ordinary code. Separating these responsibilities improves reproducibility, cost control, and auditability.

7.2 Requirements

Use LLM inference where model judgment is genuinely needed. Perform deterministic transformations, identifiers, calculations, joins, filtering, and other mechanically reproducible operations programmatically.

7.3 Example

Ask the LLM to identify governance relationships in text. Use R or Python, not another LLM call, to count those relationships and join them to metadata.

8 Stagewise Task Decomposition

8.1 Description

Complex LLM tasks are easier to control when divided into bounded stages. Each stage should have one clear analytical responsibility and an explicit intermediate representation.

8.2 Requirements

Give each stage a well-defined input, task, and structured output. Avoid asking one model call to solve several separable analytical problems simultaneously.

8.3 Example

Stage 06A identifies proposition spans. Stage 06B receives only the admitted propositions and structures them. The second stage does not independently decide whether a proposition exists.

9 Checkpointing

9.1 Description

Checkpointing saves completed work incrementally instead of waiting until the entire pipeline finishes. A late failure therefore does not erase earlier successful work.

9.2 Requirements

Persist validated outputs durably at meaningful processing-unit boundaries as soon as those units are completed.

9.3 Example

After processing and validating each document, save its result before starting the next document.

10 Resumable Execution

10.1 Description

A resumable pipeline can continue after interruption without repeating successful model calls whose persisted results are still valid.

10.2 Requirements

Detect valid completed work and reuse it. When possible, reconstruct or finalize downstream state locally instead of paying for the same inference again.

10.3 Example

If processing fails on document 47 after documents 1-46 were saved successfully, the next run begins with document 47 rather than document 1.

11 Dependency-Aware Incremental Processing

11.1 Description

The existence of an old output does not prove that it is still valid. Incremental processing determines reuse from the dependencies that produced the result.

11.2 Requirements

Track relevant inputs such as source content, prompts, configuration, model choices, schemas, and upstream artifacts. Recompute only outputs whose dependencies changed.

11.3 Example

If a prompt changes, rerun the outputs that depended on that prompt while reusing unaffected outputs.

12 Atomic Persistence

12.1 Description

Atomic persistence prevents interrupted writes from leaving partial files that appear to be completed outputs.

12.2 Requirements

Do not mark a processing unit complete until its full validated result has been durably written. Use write patterns that prevent incomplete state from becoming authoritative.

12.3 Example

Write a result to a temporary file first. After the write succeeds, atomically rename it to the final filename.

13 Artifact Immutability

13.1 Description

Completed or archival artifacts should remain trustworthy evidence of earlier runs. Later executions should not silently modify or replace them.

13.2 Requirements

Protect authoritative artifacts from overwrite and write new executions to distinct destinations when prior results must be preserved.

13.3 Example

Keep validated 2024 outputs frozen and write a new model run to a separate timestamped directory.

14 Failure Observability

14.1 Description

When a pipeline fails, the user should immediately understand where it stopped and what the failure means for already completed work.

14.2 Requirements

Report the failed processing unit, what work remains valid and preserved, and where a resumed execution should continue.

14.3 Example

Report:

```
---

PIPELINE FAILED

Failure
  Stage:          Document processing
  Processing unit: Document 47 of 100
  Error:          Invalid model response

Preserved work
  Completed units: Documents 1-46
  Outputs preserved: Yes
  Reprocessing needed: No

Recovery
  Resume at:      Document 47
  Previous results: Will be reused

EXECUTION STOPPED
Documents 1-46 remain valid.
Restart will resume at document 47.

---
```

15 Completion Reporting

15.1 Description

Regardless of whether a script involves LLMs, successful termination should be explicit. A script should not leave the user guessing whether it finished or where its outputs went.

15.2 Requirements

At successful termination, report completion and identify any externally visible files or other artifacts that were created, updated, or otherwise produced.

15.3 Example

Report:

```
---  
  
PROCESSING COMPLETE  
  
Outputs  
  Created:           outputs/results.csv  
  
COMPLETED SUCCESSFULLY  
1 output artifact was written.  
  
---
```

16 Explicit Function Provenance

16.1 Description

Function calls should make their provenance immediately legible. Language-native functions, functions supplied by external dependencies, and functions defined by the project should be visually distinguishable at the call site.

16.2 Requirements

In R, call functions from non-base packages with explicit namespace qualification using `package::function()`. In Python, prefer module-qualified calls such as `pandas.read_csv()` or `numpy.mean()` for external packages rather than importing package functions directly into the local namespace. Functions created within the project working directory must use `UPPER_SNAKE_CASE()` consistently wherever they are defined or called. Apply this convention when creating functions and when renaming functions as part of otherwise authorized work; do not rename existing project-defined functions solely to enforce the convention unless a

project-wide naming migration is explicitly authorized. Do not rename functions supplied by external packages to satisfy the project-defined naming convention.

16.3 Example

In R:

```
mean(x)
dplyr::filter(data, year >= 2020)
jsonlite::fromJSON("config.json")
VALIDATE_OUTPUT(result)
WRITE_ARTIFACT(result, path)
```

In Python:

```
import pandas
import numpy

pandas.read_csv(path)
numpy.mean(values)
VALIDATE_OUTPUT(result)
WRITE_ARTIFACT(result, path)
```

17 Standardized Script Presentation

17.1 Description

R and Python scripts should follow a consistent documentation and formatting standard without changing their computational behavior. The project's Pretty Scripts specifications provide the authoritative presentation conventions for these languages.

17.2 Requirements

After substantive implementation and validation, format R scripts according to <https://github.com/ealvaradomena/my-prompts/blob/main/prompts/pretty-r-scripts.md> and Python scripts according to <https://github.com/ealvaradomena/my-prompts/blob/main/prompts/pretty-python-scripts.md>. Treat these specifications as documentation and presentation passes, not as authorization to refactor behavior. Preserve logic, names, paths, inputs, outputs, APIs, side effects, and execution semantics unless a

separate instruction explicitly authorizes substantive changes. Revalidate behavior after applying the formatting pass.

17.3 Example

A workflow first implements and validates an R script, then applies the Pretty R Scripts specification, and finally reruns the relevant validation to confirm that only documentation and presentation changed.

18 Explicit Script I/O Contracts

18.1 Description

Each script should state its filesystem dependencies and products at the top of the file so that its role in the project pipeline can be understood without reverse-engineering the script body.

18.2 Requirements

Inspect the project structure and relevant source files across the working directory as needed to determine each script's runtime filesystem dependencies and products. Do not recursively inspect generated outputs, dependency environments, version-control internals, caches, archives, or unrelated files unless they are necessary to resolve the script's I/O behavior. At the top of every applicable script, include explicit `INPUT` and `OUTPUT` sections. Under `INPUT`, list every file within the working directory that the script reads, loads, sources, imports as project code, or otherwise depends on at runtime. Under `OUTPUT`, list every file within the working directory that the script creates or overwrites. Express each entry as a path relative to the working-directory root. Resolve constructed paths when they can be determined reliably; when a path is genuinely dynamic, document the narrowest accurate relative path pattern or location rather than inventing a filename. Keep these sections synchronized whenever filesystem behavior changes.

18.3 Example

An R script might begin with:

```
# INPUT
# - data/raw/articles.csv
# - config/model.yml
# - R/helpers.R
#
# OUTPUT
# - data/processed/articles_clean.rds
# - logs/cleaning-summary.json
```

A Python script might begin with:

```
# INPUT
# - data/processed/articles_clean.rds
# - config/extraction.json
#
# OUTPUT
# - results/extractions.jsonl
# - logs/extraction-summary.json
```